

PARKSMART

UCS 503 Software Engineering Project Report Mid-Semester Evaluation

Submitted by:

404NotFound

(102303861) Kangan Arora

(102303864) Krishiv Goyal

(102303867) Bedangi Saha

(102303873) Arnav Jain

BE Third Year, COE/CSE

Group No: 3C62

Submitted to:

Dr. Raghav B. Venkataramaiyer



Computer Science and Engineering Department
TIET, Patiala

TABLE OF CONTENTS

S.No.	Assignment	Page No.
1. • 1.1 • 1.2	Project Selection Phase • Software Bid • Project Overview	1-6 7
2. • 2.1 2.1.1 2.1.2 • 2.2 • 2.3 2.3.1 2.3.2 2.3.3 • 2.4 • 2.5	Analysis Phase • Use Cases Use-Case Diagrams Use-Case Templates • Activity Diagram and Swimlane Diagrams • Data Flow Diagrams (DFDs) DFD Level 0 DFD Level 1 DFD Level 2 • Software Requirement Specification in IEEE Format • User Stories and Story Cards	8 8-13 14-15 16 16 17 18-27 28-33

1) Project Selection Phase

1.1) Software Bid

PARKSMART

Presented By :-
KANGAN-102303861
KRISHIV GOYAL-102303864
BEDANGI SAHA-102303867
ARNAV JAIN-102303873



PROBLEM

Managing parking manually often leads to confusion, delays, and errors. Users face problems such as forgetting their parking duration, difficulties in tracking multiple vehicles, and lack of transparency in billing.

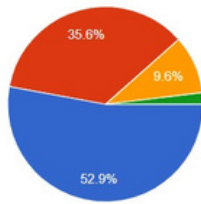
SOLUTION

This project is a **Smart Parking Management System**. It allows users to sign up or log in and register their vehicle license plates. The system records the start time when a car is parked, calculates the bill based on duration, and updates the status in real time. It provides a lightweight and user-friendly solution for basic parking automation.

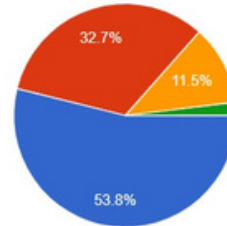
NEED STATEMENT

We are creating a SmartParking Management System
For users who want a simple and efficient way to manage their parking
That does secure signup/login, allows users to register multiple vehicle license plates, automatically tracks parking start and end times, and generates parking bills based on duration
By using Flask as the backend framework, MongoDB for lightweight data storage, and HTML templates for user-friendly interfaces.

People preferring an automated billing system

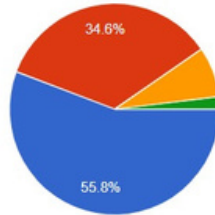


People facing parking and billing problem

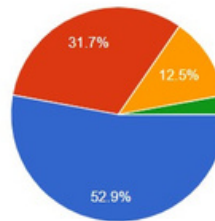


SURVEY RESULTS

People faced unfair billing due to slow cashier, congestion



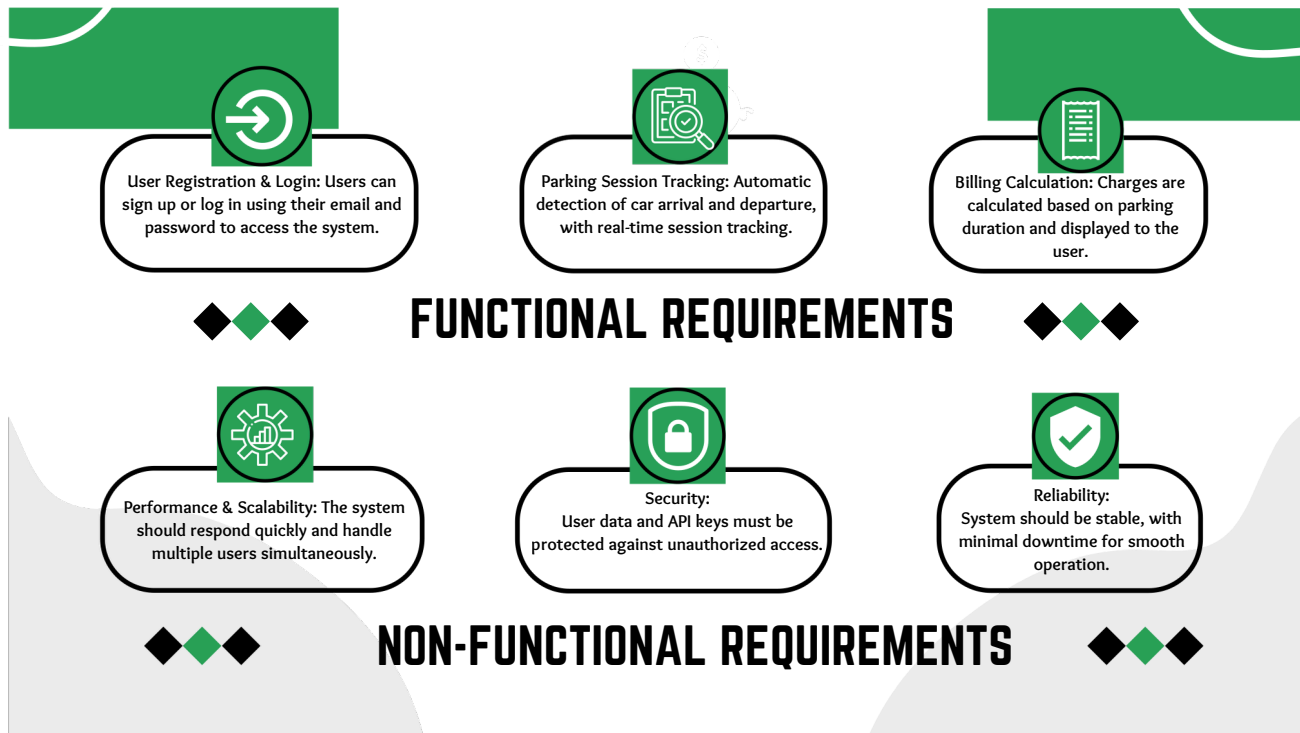
People likely to use our system



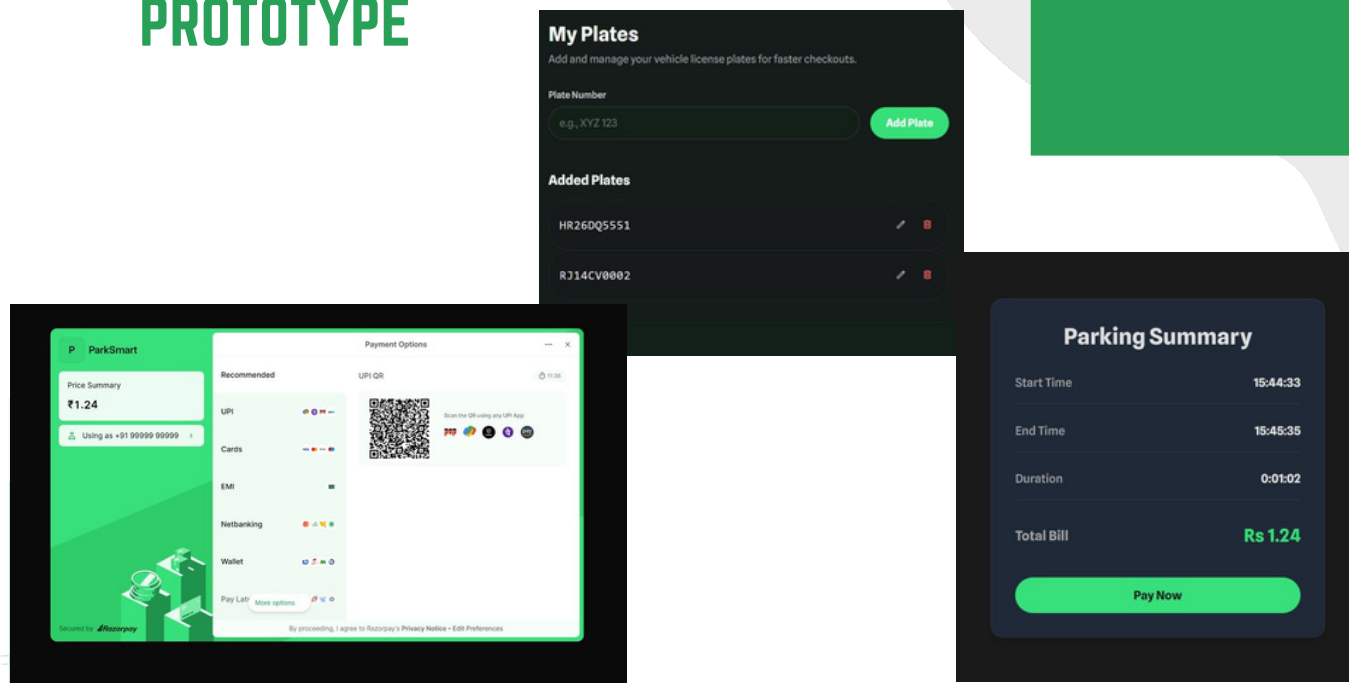
ACCESSIBILITY



- The system is built with a HTML frontend, ensuring a responsive and modern user experience across devices. Flask backend
- provides a lightweight yet powerful API for seamless communication between the client and server. MongoDB database
- ensures scalable and efficient storage of user details, license plates, and parking history. The web-based architecture
- makes it accessible from anywhere, requiring only an internet connection. The system is designed to be easy to use,
- allowing quick login, plate registration, and parking management.



PROTOTYPE



FEASIBILITY

Time	Image	License Plates
Oct. 28, 2025, 7:01 p.m.		KL07AH9981 99.60%
Oct. 28, 2025, 7:01 p.m.		KL07AH9981 99.90%
Oct. 28, 2025, 6:53 p.m.		RJ14CV0002 99.90%
Oct. 28, 2025, 6:53 p.m.		RJ14CV0002 99.90%
Oct. 28, 2025, 6:52 p.m.		HR26DQ5551 97.70%
Oct. 12, 2025, 5:26 p.m.		HR26DQ5551 99.40%

Car Plate Scanner



Backend: Detected Plate: RJ14CV0002

Car Plate Scanner



Backend: Detected Plate: KL07AH9981, Plate not registered

EVALUATION

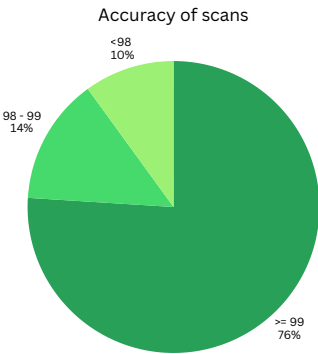
You have reached 100.0% of your Snapshot Cloud Plan quota.

Snapshot Cloud ⓘ

API Token: 500a3d6beb83762f839fae49ffff74725ea44fb26 ⓘ

Reset Date	API Calls	Max Calls/Sec	MMC	Blur	Tags
2025-11-06 11:03 AM	50 / 50	1 / second			Manage

Some countries (e.g. Egypt, United Arab Emirates, Japan, Korea (the Republic of), Thailand, Germany) are only supported with Snapshot SDK and Stream. [View supported countries.](#)



KEY FEATURES

1. User Management: Register and login using email and password
 2. password
 3. Parking Tracking: Automatic start/stop of parking sessions.
 4. sessions.
 5. Billing: Calculates cost based on parking duration.
 6. Vehicle Management: Add, edit, and remove multiple car plates.
- Accessibility: Simple, mobile-friendly interface for all users.
- Security: Safe storage of user data with scalable backend.

THANK YOU

1.2) Project Overview

The ParkSmart system is an automated parking management solution designed to simplify the vehicle entry, exit, and billing process in garages or parking facilities. The primary goal of this project is to eliminate manual ticketing and monitoring by using Automatic Number Plate Recognition (ANPR) technology integrated with modern APIs and databases.

When a vehicle enters the parking premises, a camera or scanner captures an image of the car's number plate. This image is sent to the PlateRecognizer API, which extracts the license plate number and sends it to the system's backend. The backend records the vehicle's entry time in the MongoDB Atlas database, automatically starting a timer for the duration of the parking.

Upon exit, the same process is repeated – the scanner detects the vehicle, the API extracts the number plate again, and the system matches it with the existing entry record. The timer stops, and the backend calculates the total duration of the parking session. Based on the duration, a parking fee is computed and updated in the database.

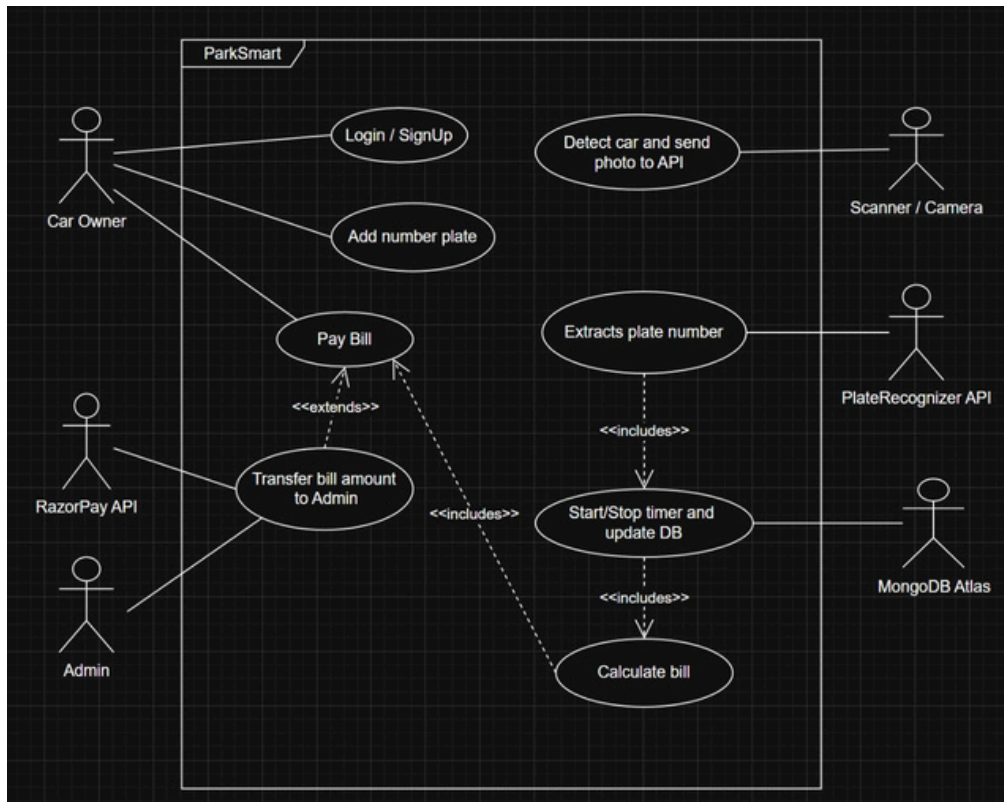
The car owner can view this amount through the ParkSmart web or mobile application, which provides a seamless interface for users to log in, register their number plates, and make payments. Payments are securely processed through the Razorpay API, which ensures reliable transaction handling and transfers the amount directly to the admin's account.

The system not only enhances user convenience but also offers real-time monitoring and efficient revenue management for parking administrators. With automation at its core, ParkSmart reduces the need for human intervention, minimizes errors, and accelerates the entire process of vehicle detection, billing, and payment.

2) Analysis Phase

2.1) Use Cases

2.1.1) Use-Case Diagrams



2.1.2) Use-Case Templates

Use Case ID :	UC01		
Use Case Name :	Login / Sign Up		
Created By :	Bedangi Saha	Last Updated By :	Bedangi Saha
Date Created :	03/11/2025	Date Last Updated :	03/11/2025

Actor :	Car Owner
Description :	This use case allows the Car Owner to register a new account or log in to the ParkSmart application using valid credentials.
Pre-conditions :	The user has installed the ParkSmart app and has internet access.
Post-conditions :	The user is successfully authenticated and redirected to the dashboard.
Frequency of use :	Frequently – every time a user accesses the webapp.
Normal course of events :	1. User opens the ParkSmart app. 2. Enters username and password or selects “Sign Up.” 3. System verifies credentials in the database. 4. If valid, the system grants access to the user dashboard.
Alternative courses :	User registers a new account using email and phone number.
Exceptions :	Invalid credentials entered → System displays error message.
Includes :	None
Assumptions :	The database and authentication API are active and connected.

Use Case ID :	UC02		
Use Case Name :	Add Number Plate		
Created By :	Bedangi Saha	Last Updated By :	Bedangi Saha
Date Created :	03/11/2025	Date Last Updated :	03/11/2025

Actor :	Car Owner
Description :	Allows a registered user to add or update their vehicle number plate in the system.
Pre-conditions :	User is logged in.
Post-conditions :	Number plate details stored in the MongoDB database.
Frequency of use :	Occasional – only when adding or updating a new vehicle.
Normal course of events :	1. User navigates to “Add Number Plate.” 2. Enters vehicle number. 3. System validates format. 4. System updates the database with the new number plate.
Alternative courses :	User updates an existing number plate instead of adding a new one.
Exceptions :	Invalid number format or DB connectivity issues.
Includes :	None
Assumptions :	User enters genuine and registered vehicle numbers.

Use Case ID :	UC03		
Use Case Name :	Detect Car and Send Photo to API		
Created By :	Bedangi Saha	Last Updated By :	Bedangi Saha
Date Created :	03/11/2025	Date Last Updated :	03/11/2025

Actor :	Scanner / Camera
Description :	Detects an entering or exiting car and sends its image to the PlateRecognizer API.
Pre-conditions :	Camera and sensors are active and configured.
Post-conditions :	Image successfully sent to the PlateRecognizer API.
Frequency of use :	Each time a vehicle enters or exits the parking area.
Normal course of events :	1. Car approaches entry or exit gate. 2. Camera captures vehicle image. 3. Image sent to the PlateRecognizer API for analysis.
Alternative courses :	Manual upload in case of camera failure.
Exceptions :	Low visibility, hardware malfunction, or API request timeout.
Includes :	None
Assumptions :	The API endpoint is available and responsive.

Use Case ID :	UC04		
Use Case Name :	Extract Plate Number		
Created By :	Bedangi Saha	Last Updated By :	Bedangi Saha
Date Created :	03/11/2025	Date Last Updated :	03/11/2025

Actor :	PlateRecognizer API
Description :	The PlateRecognizer API extracts the number plate from the car image and sends it to the backend.
Pre-conditions :	Image of vehicle received from the scanner.
Post-conditions :	Extracted number plate successfully sent to backend.
Frequency of use :	Every vehicle entry and exit event.
Normal course of events :	1. API receives image. 2. Extracts number plate using OCR. 3. Sends the recognized number plate to backend.
Alternative courses :	None
Exceptions :	API fails to recognize number; returns null or error.
Includes :	Start/Stop Timer and Update DB
Assumptions :	Number plate is clearly visible.

Use Case ID :	UC05		
Use Case Name :	Start/Stop Timer and Update DB		
Created By :	Bedangi Saha	Last Updated By :	Bedangi Saha
Date Created :	03/11/2025	Date Last Updated :	03/11/2025

Actor :	MongoDB Atlas
Description :	The backend starts or stops the parking timer and updates the database with time and vehicle details.
Pre-conditions :	Recognized number plate received from API.
Post-conditions :	Database contains updated entry and exit times.
Frequency of use :	Every car entry and exit.
Normal course of events :	1. Backend identifies car entry or exit. 2. Starts or stops the timer. 3. Updates MongoDB with time logs.
Alternative courses :	None
Exceptions :	Database connection failure or duplicate entries.
Includes :	Calculate Bill
Assumptions :	Server clock is correctly synchronized.

Use Case ID :	UC06		
Use Case Name :	Calculate Bill		
Created By :	Bedangi Saha	Last Updated By :	Bedangi Saha
Date Created :	03/11/2025	Date Last Updated :	03/11/2025

Actor :	MongoDB Atlas (Backend)
Description :	Calculates parking charges based on duration of stay.
Pre-conditions :	Entry and exit times recorded.
Post-conditions :	Bill amount generated and displayed to the user.
Frequency of use :	Every vehicle exit event.
Normal course of events :	1. System retrieves entry and exit times. 2. Computes total parking duration. 3. Applies rate per hour. 4. Displays payable amount.
Alternative courses :	None
Exceptions :	Missing timestamps or inconsistent data.
Includes :	None
Assumptions :	Pricing algorithm is predefined.

Use Case ID :	UC07		
Use Case Name :	Pay Bill		
Created By :	Bedangi Saha	Last Updated By :	Bedangi Saha
Date Created :	03/11/2025	Date Last Updated :	03/11/2025

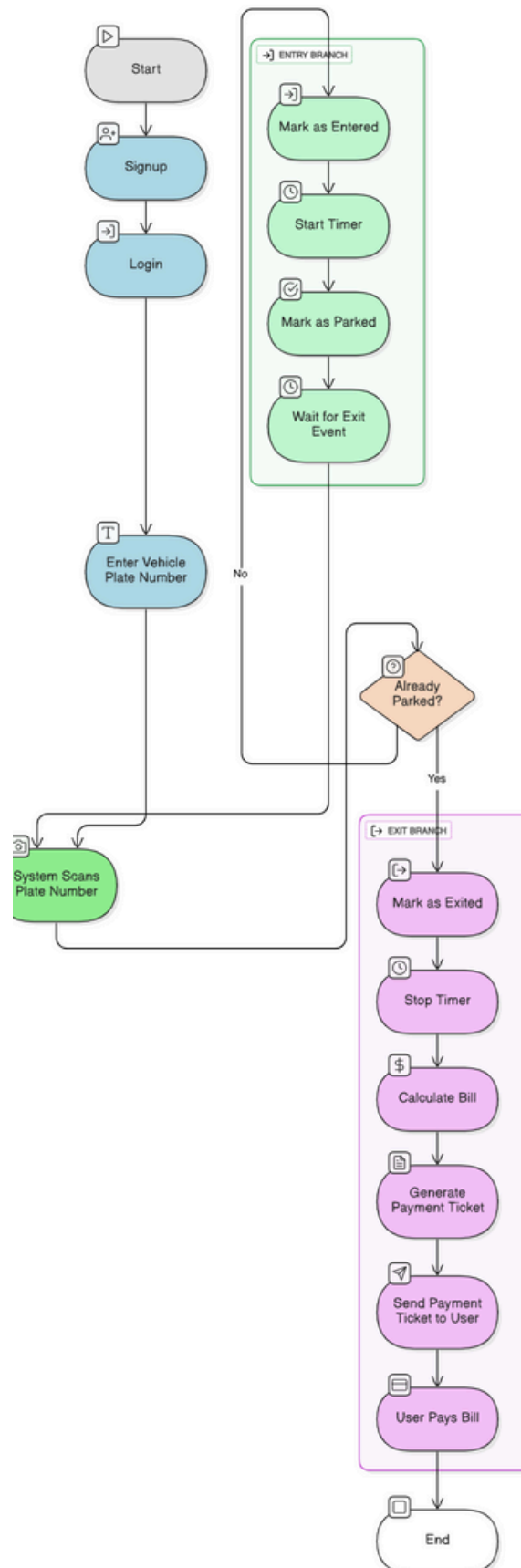
Actor :	Car Owner
Description :	Allows user to pay the generated parking bill securely through Razorpay.
Pre-conditions :	Bill calculated and displayed on user dashboard.
Post-conditions :	Payment status recorded in database.
Frequency of use :	Every exit transaction.
Normal course of events :	1. User clicks “Pay Bill.” 2. System connects to Razorpay. 3. User completes payment. 4. Razorpay confirms success to backend.
Alternative courses :	Retry payment if failed.
Exceptions :	Payment gateway timeout or transaction failure.
Includes :	Transfer Bill Amount to Admin
Assumptions :	Razorpay API is functional.

Use Case ID :	UC08		
Use Case Name :	Transfer Bill Amount to Admin		
Created By :	Bedangi Saha	Last Updated By :	Bedangi Saha
Date Created :	03/11/2025	Date Last Updated :	03/11/2025

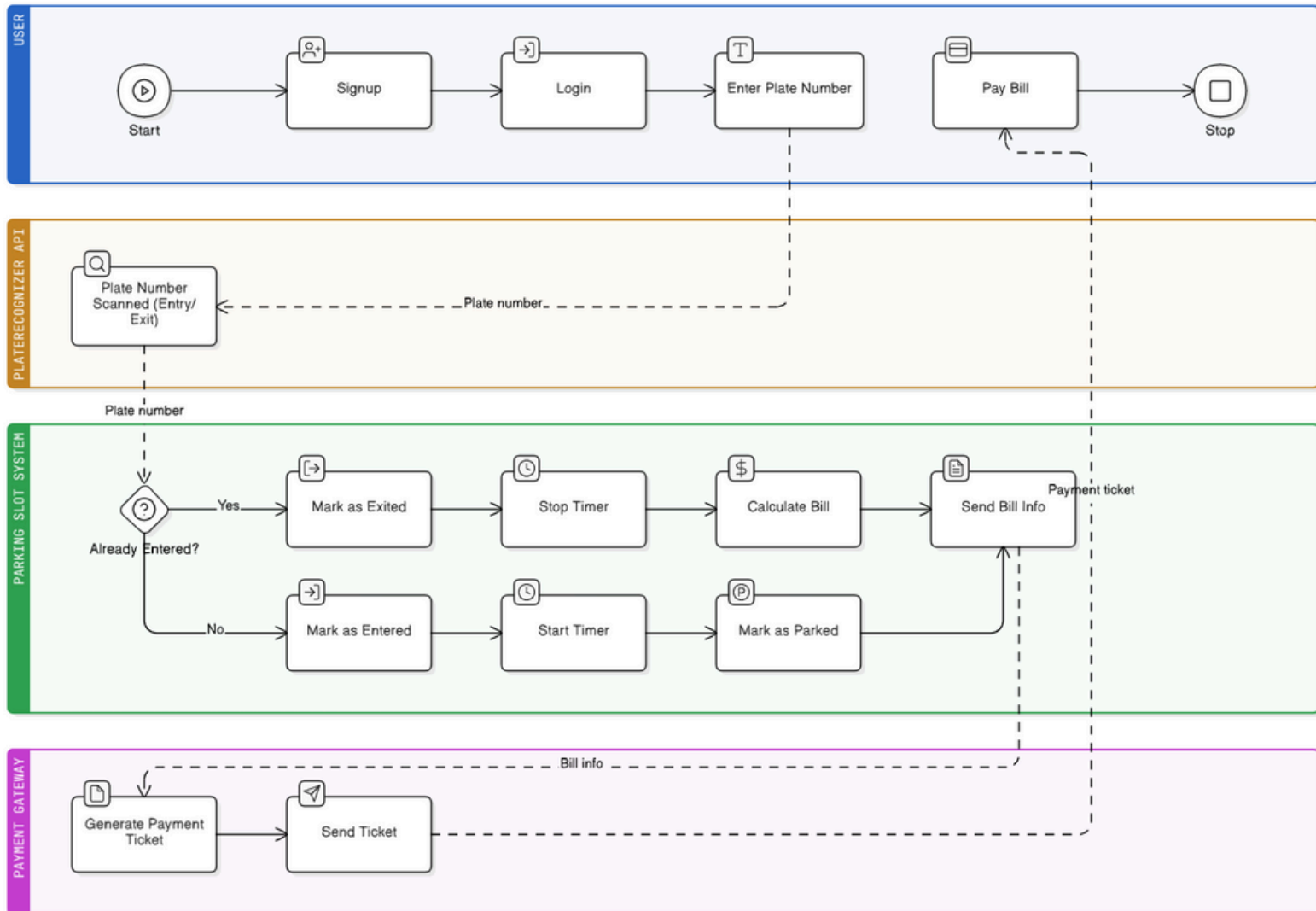
Actor :	RazorPay API, Admin
Description :	Transfers payment received from the user to the Admin’s account after successful billing.
Pre-conditions :	Payment completed by the user.
Post-conditions :	Funds transferred and recorded in admin ledger.
Frequency of use :	Each completed payment transaction.
Normal course of events :	1. Razorpay confirms payment success. 2. System triggers fund transfer to admin account. 3. Database logs payment transaction.
Alternative courses :	Manual verification by admin if auto-transfer fails.
Exceptions :	Transaction API failure or insufficient permissions.
Includes :	None
Assumptions :	Admin account is verified and linked with Razorpay.

2.2) Activity Diagram and Swimlane Diagrams

Activity Diagram

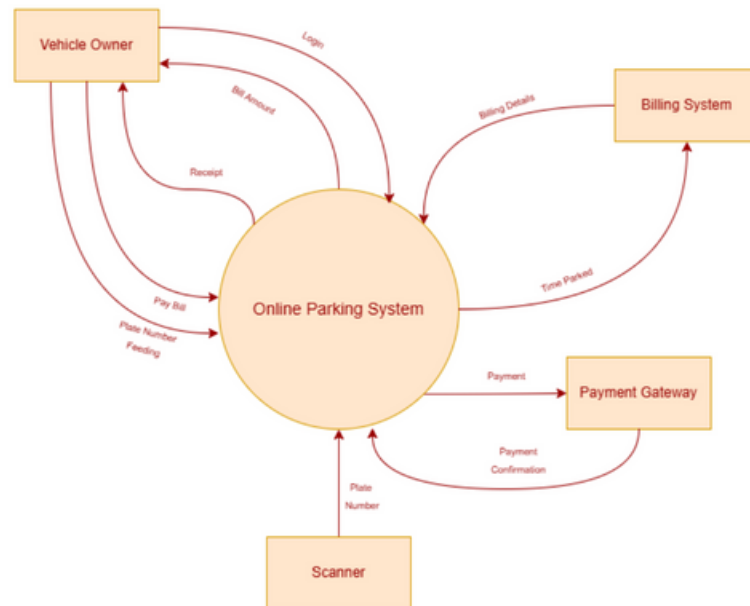


Swimlane Diagram

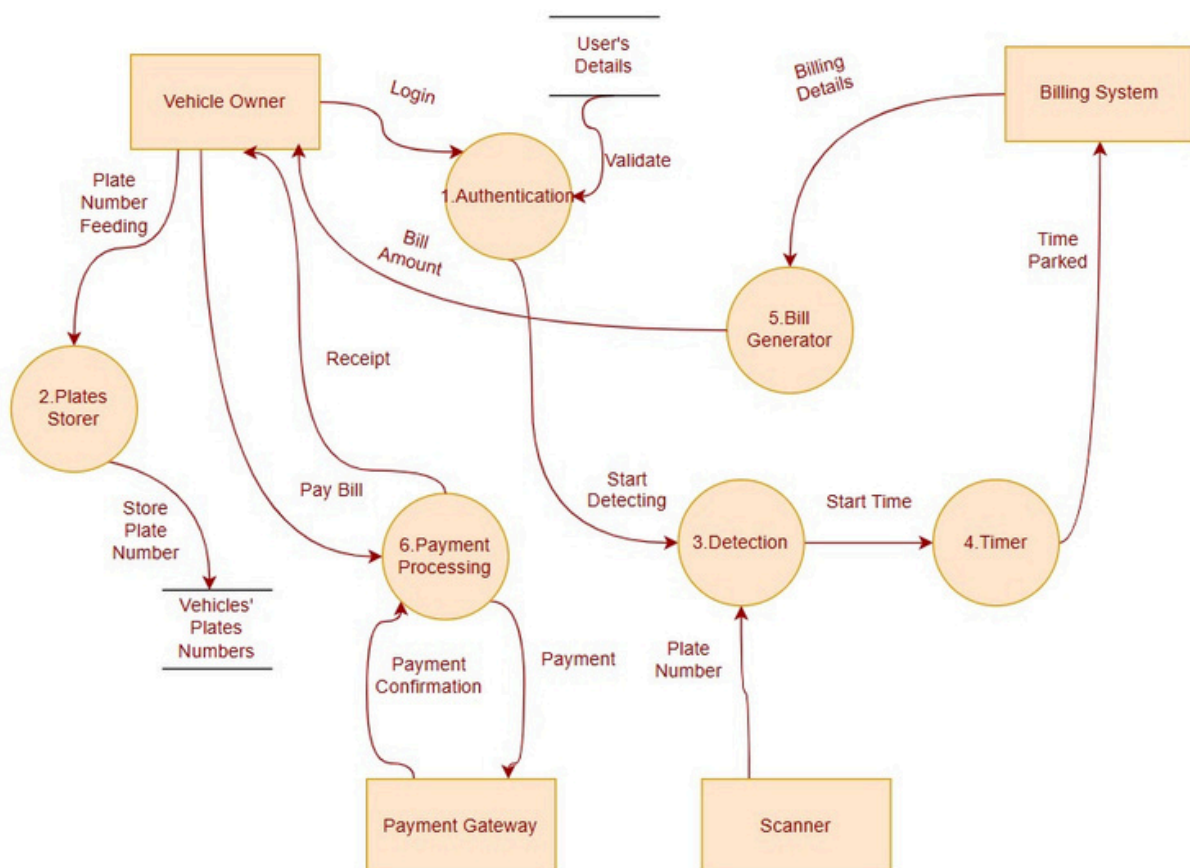


2.3) Data Flow Diagrams (DFDs)

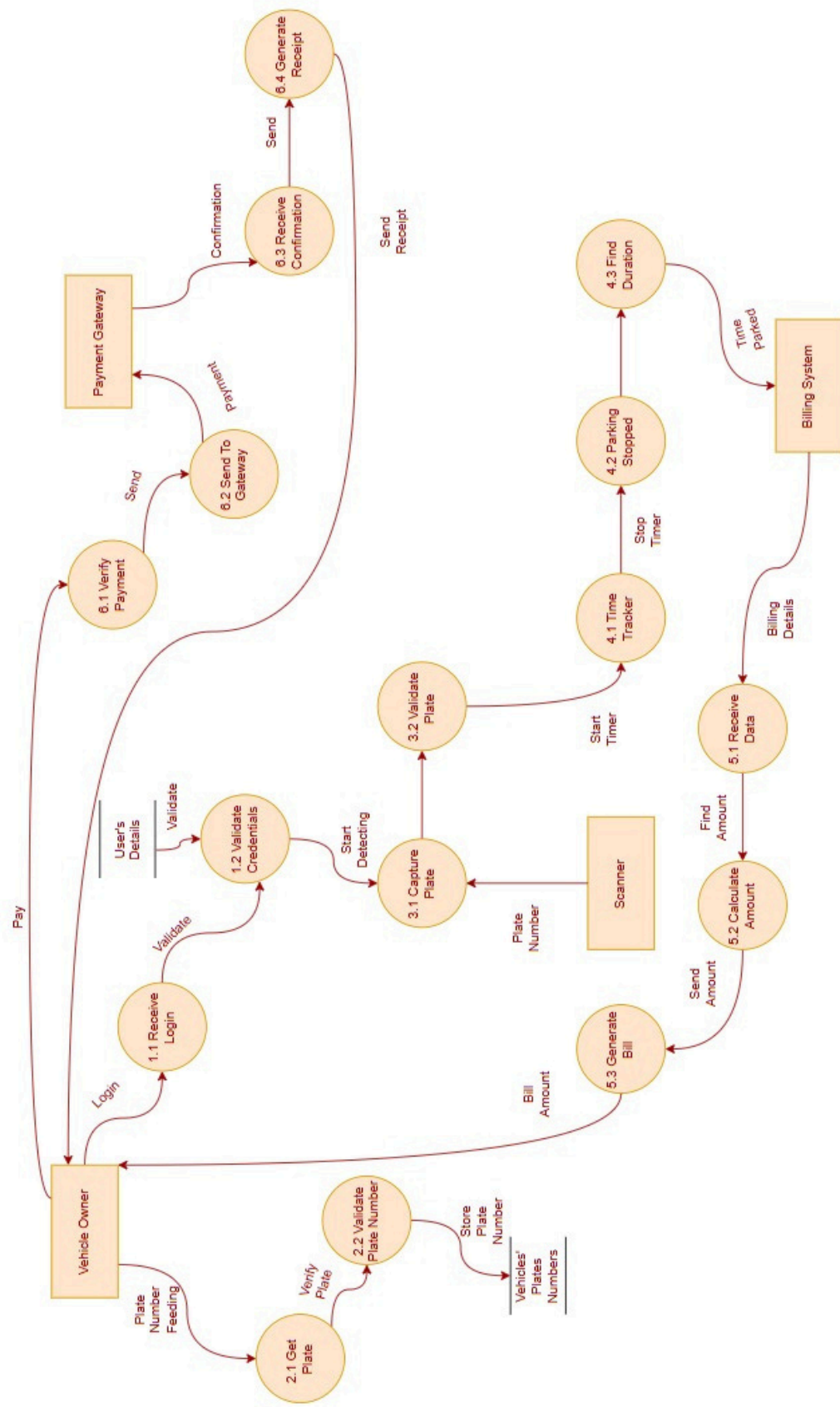
2.3.1) DFD 0



2.3.2) DFD 1



2.3.3) DFD 2



2.4) Software Requirement Specification in IEEE Format

1. Introduction

1.1 Purpose

The purpose of this Software Requirements Specification (SRS) document is to define a comprehensive and detailed description of the Online Smart Parking Management System (OSPMS). The document outlines the functional and non-functional requirements that guide the system's design, implementation, and validation processes. This SRS serves as a formal reference for developers, testers, system architects, and stakeholders to ensure that all project participants share a unified understanding of the system's objectives and scope. It provides a foundation for planning, development, testing, and maintenance activities. The document ensures that all system requirements — from vehicle detection and identification to automated billing and user account management — are precisely defined to achieve a reliable, efficient, and user-friendly smart parking solution.

1.2 Scope

The Online Smart Parking Management System (OSPMS) aims to automate the entire parking process — from vehicle entry to exit — using modern IoT and AI-based technologies. The system utilizes Automatic Number Plate Recognition (ANPR) to detect and identify vehicles in real-time through installed cameras at entry and exit points. Once a vehicle is detected, the system automatically:

- Captures and stores the vehicle's license plate number.
- Allocates an available parking slot.
- Records the entry time of the vehicle.
- Begins automated billing in the corresponding user account.

When the vehicle exits, the system again uses ANPR to verify the vehicle, calculate the total parking duration, and generate a digital bill based on pre-defined parking rates. The user can review and settle payments online through integrated payment gateways such as Razorpay or UPI.

Additionally, the system maintains a user-friendly dashboard where users can:

- View their parking history and payment records.
- Reserve slots in advance (if applicable).
- Manage personal details and linked payment methods.

The OSPMS minimizes manual effort, reduces waiting time at parking lots, and eliminates traditional ticket-based systems, providing a smart, secure, and seamless digital parking experience for users and administrators alike.

1.3 Definitions, Acronyms, and Abbreviations

Term / Acronym	Definition
API	Application Programming Interface — a set of protocols enabling communication between software components.
ANPR	Automatic Number Plate Recognition — a technology that uses image processing to identify vehicle license plates.
DB	Database — a structured repository used to store all system data such as user details, vehicle logs, and transaction history.
UI	User Interface — the graphical interface through which users interact with the system.
SRS	Software Requirements Specification — this document defining the system's requirements.
UPI	Unified Payments Interface — a digital payment system that allows instant money transfer via mobile applications.
Razorpay	A secure online payment gateway used for transaction processing within the system.

1.4 References

- IEEE Standard for Software Requirements Specifications (IEEE Std 830-1998).
- Flask Framework Documentation -<https://flask.palletsprojects.com>.
- Razorpay Developer API Documentation-<https://razorpay.com/docs/api>.
- Plate Recognizer API Documentation <https://platerecognizer.com>.
- Python 3 Official Documentation –<https://docs.python.org/3>.

1.5 Overview

This document presents a structured overview of the Online Smart Parking Management System, including its objectives, features, design constraints, user roles, and interface requirements. It defines how the system detects vehicles, processes data, manages billing, and handles payments efficiently. Subsequent sections of this SRS elaborate on:

- Overall system description, including user characteristics and design considerations.
- Functional requirements, detailing each system module such as vehicle detection, billing, payment, and reporting.

- Non-functional requirements, addressing performance, scalability, security, and usability expectations.
- System interfaces, defining interactions between users, hardware components (e.g., cameras), and external APIs (e.g., Razorpay, Plate Recognizer).

By providing a clear, detailed, and structured specification, this document serves as the cornerstone for the system's successful development, testing, and deployment.

2. Overall Description

2.1 Product Perspective

The Online Smart Parking Management System (OSPMS) is a web-based and cloud-integrated platform designed to automate vehicle parking operations using AI-driven number plate recognition and digital payment processing. The system follows a modular architecture integrating multiple technologies to ensure smooth communication between different layers of the application:

- Frontend: Developed using HTML / CSS, providing a responsive and intuitive interface accessible through both desktop and mobile browsers.
- Backend: Built using Flask (Python), responsible for handling server-side logic, API integration, and communication between components.
- Database: Utilizes MongoDB, a NoSQL database for efficiently storing user accounts, vehicle records, parking sessions, and transaction histories.
- Third-Party APIs:
 - Plate Recognizer API – for automatic number plate detection and recognition.
 - Razorpay Payment Gateway – for processing secure online payments and generating digital receipts.

The system operates as an independent web application, but it can also be extended or integrated with IoT sensors, smart barriers, and parking guidance systems in the future. It provides both user-side and admin-side functionalities to ensure complete parking automation and management.

2.2 Product Functions

The core functional modules of the OSPMS include:

- User Registration and Authentication: Secure user sign-up and login mechanisms using email and password credentials, ensuring authorized access.
- Vehicle License Plate Management: Allows users to add, edit, or remove vehicle license plates linked to their account for automated identification during entry/exit.
- Real-Time Vehicle Detection via ANPR: Cameras installed at parking entry and exit points automatically detect and recognize vehicle plates using the Plate Recognizer API.
- Automated Parking Session and Billing: When a vehicle enters, the system records the timestamp and starts tracking parking duration. Upon exit, billing is automatically calculated based on elapsed time and applicable rates.

- **Online Payment and Receipt Generation:** Integration with Razorpay enables secure digital payments through UPI, cards, or wallets. Once payment is successful, a digital receipt is generated and stored in the user's account.

Collectively, these modules ensure a fully automated, paperless, and efficient parking experience, eliminating the need for manual ticketing and reducing human intervention.

2.3 User Characteristics

The system is designed for two primary user groups:

- **General Users (Vehicle Owners):** Individuals who use parking facilities. They can register, manage their vehicles, track parking sessions, and make online payments. Basic familiarity with smartphones or web browsers is sufficient.
- **Parking Administrators / Managers:** Personnel responsible for monitoring parking operations. They can oversee vehicle entries/exits, manage billing logs, and handle issues such as payment disputes or manual overrides.

The user interface (UI) is designed to be intuitive, responsive, and accessible, ensuring that users with minimal technical knowledge can operate the system effectively.

2.4 General Constraints, Assumptions, and Dependencies

Constraints:

- The system requires a stable internet connection for real-time data synchronization and API communication.
- The ANPR function depends on camera image quality and ambient lighting for accurate plate recognition.
- The Razorpay sandbox mode is used during testing; production mode is deployed for live environments.

Assumptions:

- Users will provide accurate and valid information during registration.
- Vehicles will have clear and legible license plates compatible with ANPR standards.
- The parking area has properly installed cameras at entry and exit gates.

Dependencies:

- Plate Recognizer API for real-time plate detection.
- Razorpay API for payment gateway operations.
- MongoDB service availability for data storage.
- Flask web server for backend service handling.

2.5 Apportioning of Requirements

Some features are planned for future development and enhancement phases to improve system intelligence and scalability, including:

- **Predictive Parking Availability:** Real-time analytics to predict available slots based on historical trends and live occupancy.
- **AI-Based Slot Optimization:** Automatic slot allocation to minimize congestion and maximize utilization.
- **Mobile Push Notifications:** Real-time updates for entry, exit, billing, and available slot alerts via mobile app integration.
- **Integration with Smart Barriers and IoT Sensors:** For automatic gate operation and environment monitoring.

2.6 User Stories and Story Cards

User Signup

As a user, I want to sign up using my email and password, so I can create an account and access the parking system.**Confirmation:**

- **Success:**
 1. User information stored in MongoDB.
 2. Redirected to the login page with a success message.
- **Failure:**
 1. Email already exists in the system.
 2. Invalid email format or weak password.
 3. Database or service unavailable.

User Login

As a user, I want to log in using my credentials, so I can manage my vehicles and view my parking details.**Confirmation:**

- **Success:**
 1. Credentials verified; user redirected to dashboard.
 2. "Remember me" enables persistent login.
- **Failure:**
 1. Invalid credentials or unregistered email.
 2. Service temporarily unavailable.

Vehicle Management

As a user, I want to add, edit, or remove my vehicle details so that the system can recognize my car automatically.**Confirmation:**

- **Success:**
 1. Vehicle successfully added, modified, or deleted.
 2. Updated vehicle list visible on user dashboard.
- **Failure:**
 1. Duplicate or invalid plate entry.

2. Database or API error.

Parking Session Tracking

As a user, I want the system to automatically detect my vehicle's entry and exit using ANPR and display live session status.**Confirmation:**

- **Success:**
 1. Vehicle entry and exit successfully detected.
 2. Real-time updates shown in the user's dashboard.
- **Failure:**
 1. Detection error (camera failure or poor lighting).
 2. Session not found or incomplete data.

Billing and Payment

As a user, I want my parking charges to be calculated automatically and paid securely online through Razorpay.**Confirmation:**

- **Success:**
 1. Bill generated based on parking duration.
 2. Payment processed through UPI, card, or wallet.
 3. Digital receipt generated and saved in user history.
- **Failure:**
 1. Payment declined due to insufficient funds or network issues.
 2. Temporary unavailability of payment gateway services

3. Specific Requirements

3.1 External Interface Requirements

3.1.1 User Interface Requirements

The system provides a responsive, browser-based web interface designed using HTML/CSS/JS, ensuring accessibility across desktops, tablets, and smartphones. The major user interface components include:

- **Signup Page:** Allows new users to register using email, password, and optional vehicle details.
- **Login Page:** Authenticates users with secure credential validation and "Remember Me" functionality.
- **Dashboard:** Displays an overview of active parking sessions, billing details, and transaction history.
- **Vehicle Management Page:** Enables users to add, edit, and remove registered vehicle license plates.

- Billing and Payment Page: Shows generated bills and provides Razorpay-integrated payment options.
- Admin Interface (optional): Allows parking managers to monitor entry/exit logs, payment reports, and manage user records.

All pages follow a consistent design pattern, featuring a navigation bar, clear action buttons, real-time notifications, and responsive layouts optimized for different devices.

3.1.2 Hardware Interface Requirements

The system depends on the following hardware components for its functionality:

- Cameras / IP-based ANPR Systems: Installed at parking entry and exit points to capture vehicle images for number plate recognition.
- Server Hardware: Capable of running Flask backend services, MongoDB database, and handling concurrent API requests.
- Network Infrastructure: Stable internet connectivity for API communication, database operations, and payment gateway integration.

Recommended minimum server configuration:

- Processor: Quad-core (2.5 GHz or higher)
- RAM: 8 GB or more
- Storage: 100 GB SSD
- Operating System: Linux/Ubuntu or Windows Server

3.1.3 Software Interface Requirements

The system integrates the following software components and third-party APIs:

Component / API	Purpose	Type
Flask	Backend web framework used to develop RESTful APIs and manage server-side logic.	Backend
MongoDB	NoSQL database for storing user, vehicle, and transaction records.	Database
Razorpay API	Provides secure online payment processing and digital receipts.	Payment Gateway
Plate Recognizer API	Handles number plate detection from live camera feeds.	ANPR Service
HTML, CSS, JS	Builds a dynamic and responsive user interface.	Frontend

Each interface follows standard REST API communication protocols (HTTP/HTTPS), with all API keys and credentials securely managed via environment variables.

3.2 Detailed Description of Functional Requirements

The functional requirements define the system behavior and expected outputs under specified conditions.

1. User Authentication and Management:
 - The system must allow secure registration and login using encrypted passwords.
 - Users can reset forgotten passwords via a secure token-based link.
 - Session tokens or cookies ensure continuous authentication.
2. Vehicle Management (CRUD):
 - Users can Create, Read, Update, and Delete (CRUD) vehicle plate records.
 - Each vehicle is uniquely linked to a user account.
 - The system validates plate formats and prevents duplicates.
3. ANPR-based Entry and Exit Detection:
 - Cameras capture vehicle images upon entry and exit.
 - The Plate Recognizer API identifies the vehicle plate and cross-verifies it with the database.
 - Entry time and exit time are automatically logged.
4. Automated Billing System:
 - Billing is computed automatically based on the total duration of the parking session.
 - The system applies predefined rate slabs (e.g., hourly or flat rate).
 - Bill details are stored under the user's transaction history.
5. Online Payment and Receipt Generation:
 - Razorpay integration allows users to pay via UPI, cards, or wallets.

- Upon successful payment, the system updates the transaction status and generates a digital receipt (PDF or downloadable format).
 - Payment failures or cancellations trigger appropriate error messages and retries.
6. User Dashboard:
- Displays current session details, previous parking history, and payment logs.
 - Includes live status updates for ongoing parking sessions.
 - Provides filtering options by date, vehicle, and payment status.

3.3 Performance Requirements

The system is designed to maintain optimal responsiveness and reliability under normal operating conditions:

- Vehicle detection and entry logging: ≤ 3 seconds per transaction.
- Bill calculation and payment confirmation: ≤ 5 seconds under normal load.
- System should support at least 100 concurrent user sessions without performance degradation.
- Database query response times must not exceed 2 seconds for standard operations.

3.4 Logical Database Requirements

MongoDB collections: • users: email, password, plates, active status, bill, timestamps. • transactions: order_id, payment_id, amount, status. Indexes ensure fast lookups by email and license plate.

3.5 Quality Attributes

- **Security:** Implements password hashing (e.g., bcrypt) and HTTPS for secure data transmission. All sensitive information, including API keys, is stored as environment variables.
- **Usability:** The interface is simple, modern, and intuitive, enabling quick navigation and minimal user input.
- **Reliability:** Designed with error handling and retry mechanisms to maintain continuous operation even under network or API failures.
- **Scalability:** Uses modular Flask services and MongoDB to support database expansion, load balancing, and future microservices.
- **Availability:** System uptime should exceed 99%, supported by database backups and automatic service recovery mechanisms.
- **Maintainability:** Follows MVC architecture and clean coding practices, making updates and debugging straightforward.

3.6 Other Requirements

- The application must comply with **standard web security protocols**, including **HTTPS, CORS, and CSRF protection**.
- All environment-specific configurations (API keys, database URLs, payment secrets) must be stored securely using **.env files**.
- The system architecture supports **containerization and deployment using Docker or cloud services** (AWS, Azure, etc.).

- Future versions may transition into a **microservice architecture** for enhanced modularity.

4. Change History

Version	Date	Description
1.0	Initial Draft	Basic structure and requirements defined.
1.1	Updated	Added User Stories and Confirmation Scenarios.
1.2	Revised	Integrated backend, database, and API details for Flask and MongoDB.
1.3	Finalized	Expanded Specific Requirements with quality and performance metrics.

2.5) User Stories and Story Cards

- As a user, I want to sign up using my email and password, so I can create an account to access the system.

#0001	Sign Up
• As a user, I want to sign up using my email and password, so I can create an account to access the system.	
Create an Account Join ParkSmart to find and manage your parking easily. Username Krishiv Email address example@example.com Password ***** Sign up Already have an account? Login	

Success – valid user registration completed successfully

- User enters valid email and password, and account is created in the database.
- Password is encrypted before storage to ensure data security.
- After successful signup, user is redirected to the dashboard page.
- Display message: "Signup successful! Please log in next time."

Failure – Display message:

- "Email address in wrong format."
- "Password too short — must be at least 8 characters."
- "Email already registered — please log in instead."
- "Unable to create account — please check your details and try again."
- "Service unavailable — please try again later."

- As a user, I want to log in with my credentials, so I can manage my parking details.

#0002	User Login
• As a user, I want to log in with my credentials, so I can manage my parking details.	
Welcome back Enter your credentials to access your account. Email address example@gmail.com Password ***** Log in Don't have an account? Create an account	

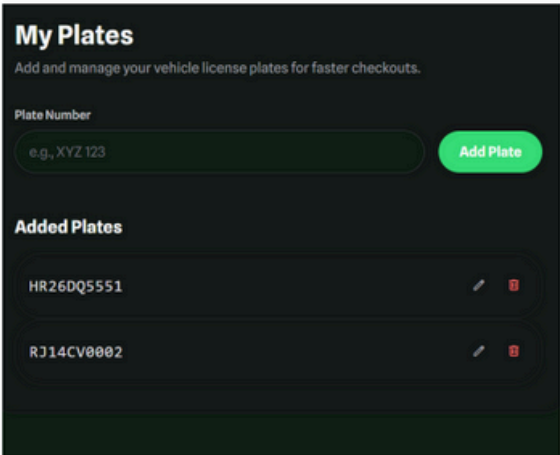
Success – valid credentials verified successfully

- User enters a registered email and correct password.
- System authenticates credentials against the database.
- On successful verification, user is redirected to the dashboard/home page.
- Display message: "Login successful! Redirecting to your dashboard."

Failure – Display message:

- "Email address in wrong format."
- "Unrecognized email — please sign up first."
- "Incorrect password — please try again."
- "Account locked due to multiple failed attempts — try again later."
- "Service unavailable — please try again later."

- As a user, I want to register my vehicle's license plate, so I can track its parking sessions automatically.
- As a user, I want to add multiple vehicles to my account, so I can manage parking for any of my cars in one place.
- As a user, I want to edit or remove a vehicle's details, so I can keep my registered vehicle list up to date.

#0003	Vehicle Management
• As a user, I want to register my vehicle's license plate, so I can track its parking sessions automatically. • As a user, I want to add multiple vehicles to my account, so I can manage parking for any of my cars in one place. • As a user, I want to edit or remove a vehicle's details, so I can keep my registered vehicle list up to date.	
	

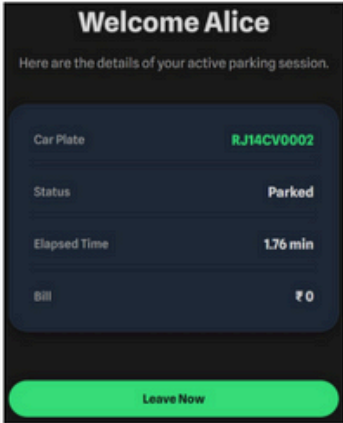
Success – valid vehicle details processed successfully.

- Vehicle registration number is verified and stored in the database.
- Display message: "Vehicle successfully added to your account."
- User can view all registered vehicles on their dashboard.
- User can add multiple vehicles without overwriting existing ones.
- On editing, updated vehicle details are reflected instantly.
- On removal, selected vehicle is deleted from user's account after confirmation.

Failure – Display message:

- "Invalid license plate format — please recheck and try again."
- "Vehicle already registered under your account."
- "Vehicle not found — unable to edit or remove."
- "Service unavailable — please try again later."
- "Unable to save changes — please check your input."

- As a user, I want the system to automatically detect when my car enters or leaves the parking, so I can avoid manually checking in or out.
- As a user, I want to see the live status of my parked vehicle (active/inactive), so I can monitor my parking in real time.
- As a user, I want the system to automatically end my session when I vacate my spot, so I can avoid extra charges.

#0004	Parking Session Tracking
• As a user, I want the system to automatically detect when my car enters or leaves the parking, so I can avoid manually checking in or out. • As a user, I want to see the live status of my parked vehicle (active/inactive), so I can monitor my parking in real time. • As a user, I want the system to automatically end my session when I vacate my spot, so I can avoid extra charges.	
	

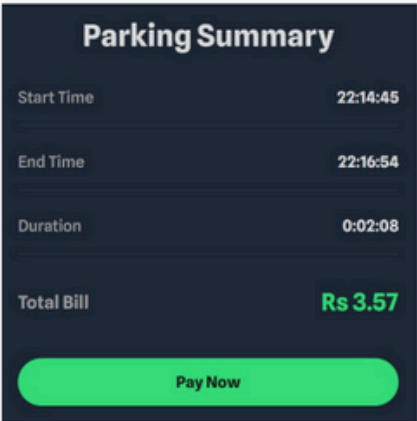
Success – parking session detected and updated correctly

- When a registered vehicle enters, the system automatically records the start time.
- Live status updates to “Active” on dashboard once parking begins.
- When the vehicle exits, end time is automatically recorded.
- Live status changes to “Inactive” after exit detection.
- Display message: “Parking session started/ended successfully.”
- System ensures accurate time tracking for billing purposes.

Failure – Display message:

- “Unable to detect vehicle entry — please check camera or sensor.”
- “Vehicle not recognized — ensure your license plate is registered.”
- “Error updating session — please try again later.”
- “Parking session already active — cannot start a new one.”
- “Session not found — cannot end parking.”
- “System temporarily unavailable — please contact support.”

- As a user, I want the system to automatically calculate my parking charges based on duration, so I can see a fair and transparent bill.
- As a user, I want to view my bill anytime, so I can track my expense and usage.

#0005	Billing
• As a user, I want the system to automatically calculate my parking charges based on duration, so I can see a fair and transparent bill. • As a user, I want to view my bill anytime, so I can track my expense and usage.	
 The image shows a mobile app interface for a parking summary. It has a dark blue background with white text. At the top, it says 'Parking Summary'. Below that, there are four rows of data: 'Start Time' with the value '22:14:45', 'End Time' with '22:16:54', 'Duration' with '0:02:08', and 'Total Bill' with 'Rs 3.57'. The 'Total Bill' is highlighted in green. At the bottom, there is a green button with the text 'Pay Now'.	

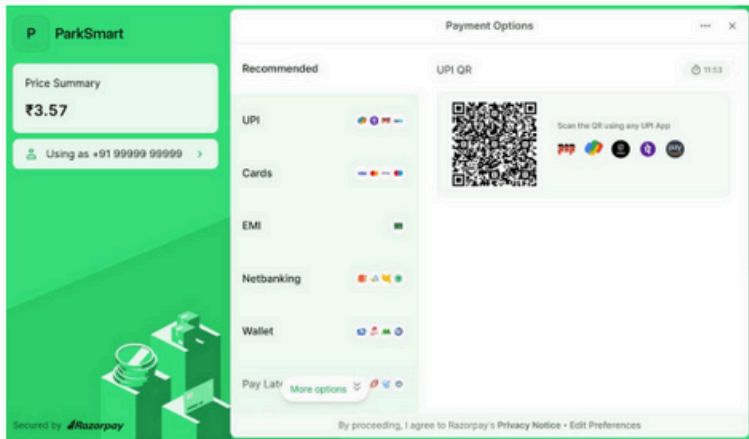
Success – billing details calculated and displayed correctly

- When a parking session ends, the system automatically calculates total charges using the rate per hour/minute and total duration.
- Display message: “Billing complete — your total charge is ₹[amount].”
- Bill is generated only after successful session completion to ensure accuracy.

Failure – Display message:

- “Unable to calculate bill — session data incomplete.”
- “Billing service unavailable — please try again later.”
- “No bill found — please check if a parking session has ended.”
- “Incorrect data — please contact support for verification.”

- As a user, I want to receive a digital bill upon exit, so I can keep a record without needing a paper receipt.
- As a user, I want to pay online through digital wallets or UPI, so I can avoid cash transactions.

#0006	Payment
• As a user, I want to receive a digital bill upon exit, so I can keep a record without needing a paper receipt. • As a user, I want to pay online through digital wallets or UPI, so I can avoid cash transactions.	
	

Success – digital billing and payment processed correctly

- Once the parking session ends, a digital bill is automatically generated and displayed on the screen.
- The bill includes details like vehicle number, duration, rate, total amount, and time of exit.
- User can choose from multiple payment options — UPI, debit/credit card, or digital wallets.
- On successful payment, display message: “Payment successful!”
- Payment status and transaction ID are stored securely in the payment app.

Failure – Display message:

- “Unable to generate bill — please try again.”
- “Payment failed — please verify your payment method.”
- “Network error — payment could not be completed.”
- “Service unavailable — please try again later.”